

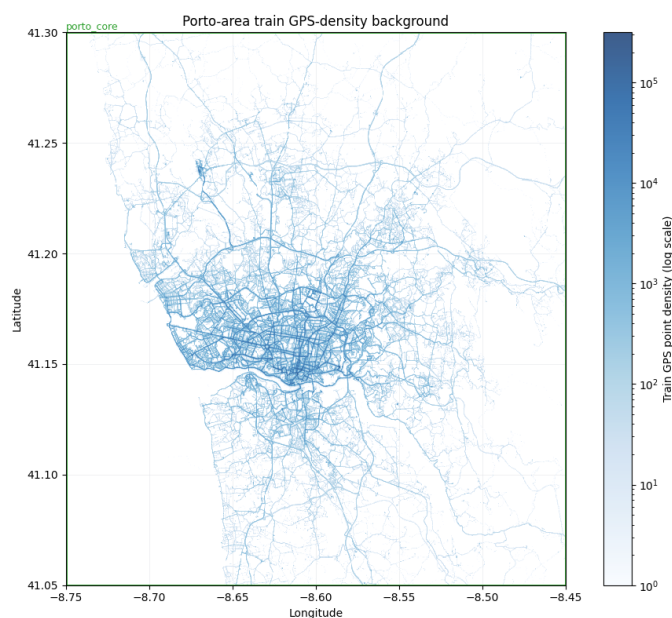
Taxi Destination Prediction

From Partial Trajectories to a Reproducible Ensemble

Project 3 – Competition

Technical report

Louis Hogge



17 July 2026

Abstract

We developed an end-to-end system for predicting the destination of a Porto taxi from a partially observed trajectory. Starting from 1,710,670 labelled trips, we retained 1,672,901 after validity, spatial, and duration filtering. We generated synthetic training prefixes with the same length distribution as the 320 supplied competition trajectories, then represented each prefix with its first and last GPS points, temporal variables, and dispatch metadata. A shared 541-feature matrix supports six model families, while CatBoost uses a 36-column representation with native categorical variables.

We compared Linear Regression, Ridge, KNN, Decision Tree, Random Forest, XGBoost, and CatBoost under a common validation design, then combined the two strongest complementary models. Random Forest achieved the best standalone results: 2.119 km on validation and 2.206 km on the 160-trip local public subset. A fixed blend of 81.399% Random Forest and 18.601% XGBoost reduced the grouped cross-fitted validation error by 6.531 m on average and achieved the best local public score, 2.191 km. The gain is consistent but small, so the main conclusion is that Random Forest provides the strongest individual model and XGBoost adds a modest corrective signal. The principal limitations are the random rather than temporal validation split, repeated model selection on one holdout, and mixed CPU/GPU runtime measurements.

Contents

1	Introduction	1
2	Data and feature construction	1
2.1	Task and data	1
2.2	Cleaning and validation split	3
2.3	Competition-aligned synthetic prefixes	3
2.4	Fixed-width representation	4
3	Models	4
3.1	Linear and neighbourhood baselines	5
3.2	Tree-based models	5
3.3	Random Forest–XGBoost ensemble	5
4	Experimental design	6
4.1	Evaluation metric	6
4.2	Model selection and final fitting	6
4.3	Grouped ensemble selection	7
4.4	Reported populations and reproducibility	7
5	Results	7
5.1	Random Forest is the strongest standalone model	7
5.2	Validation and local scoring tell a consistent top-level story	9
5.3	The ensemble gain is consistent but small	9
6	Discussion, limitations, and next steps	10
6.1	Interpretation	10
6.2	Limitations	10
6.3	Priorities for improvement	11
7	Conclusion	11
	References	12

List of Figures

1	Temporal distributions of training trips and competition prefixes	3
2	Observed prefix-length quantiles	4
3	Shared training, selection, and final-fitting workflow	6
4	Local public-score ranking	8
5	Mean ensemble gain over Random Forest by cross-fit repeat	9

List of Tables

1	Raw CSV schema	2
2	Populations and cached representations	2
3	Final model comparison	8

1 Introduction

Destination prediction from a partial vehicle trajectory is a conditional forecasting problem: the observed path shows where a journey has been, but the unobserved suffix may still contain turns, detours, or a long final leg. Accurate predictions can support taxi dispatch, demand anticipation, and passenger matching. We study this problem with the Porto taxi data from the ECML/PKDD trajectory challenge [7]. The task is to predict the final latitude and longitude of 320 trips from cut-off initial trajectories and trip metadata.

The competition metric is mean Haversine distance, so a useful model must control great-circle destination error rather than coordinate-wise error alone. Our work addresses three questions:

1. How should a variable-length partial trajectory be represented so that conventional regressors can predict a two-dimensional destination?
2. Which linear, neighbourhood-based, single-tree, bagged-tree, and boosted-tree methods perform best under a shared validation protocol?
3. Can a low-capacity blend improve the strongest standalone model without selecting its weight on the competition labels?

Our final system has four main components:

- one shared preprocessing pipeline for cleaning complete trajectories and constructing test-like partial prefixes;
- fixed-width spatial, temporal, and categorical features used consistently across model families;
- seven standalone regressors spanning simple baselines to GPU-accelerated boosting; and
- a grouped cross-fitted Random Forest–XGBoost blend selected before local competition scoring.

We use the validation holdout for model selection. After the competition labels became locally available, we also score the frozen submissions on the 160-row public subset, the complementary 160-row private subset, and all 320 trips. These local scores describe the final submissions; they are not used to learn the ensemble weight. The public mean is the primary competition-facing result, while validation, tail error, and runtime provide complementary evidence.

2 Data and feature construction

2.1 Task and data

The training set contains 1,710,670 complete Porto taxi trajectories and the test set contains 320 partial trajectories. Both CSV files use the nine-field schema in Table 1. Each GPS point is recorded every 15 seconds as [LONGITUDE, LATITUDE]; the last point of a complete training polyline is the destination. The test trips occur after the training year, so they introduce both a forecasting task and a temporal shift [7].

Field	Type	Values / scope	Meaning
TRIP_ID	String	May repeat	Trip identifier.
CALL_TYPE	Character	A, B, or C	Dispatch mode: central call, taxi stand, or street hail.
ORIGIN_CALL	Integer	Type A; may be missing	Anonymized caller identifier when a central-call trip provides one.
ORIGIN_STAND	Integer	Type B; otherwise missing	Taxi-stand identifier for stand-based trips.
TAXI_ID	Integer	442 taxis	Identifier of the taxi associated with the trip.
TIMESTAMP	Integer	Unix seconds	Trip start time.
DAY_TYPE	Character	A, B, or C	Normal day, holiday, or day preceding a holiday, respectively.
MISSING_DATA	Boolean	True or False	Whether the recorded GPS stream contains missing samples.
POLYLINE	List as string	Coordinate pairs	GPS samples stored as [LONGITUDE, LATITUDE] every 15 seconds; the final training point is the destination, whereas test rows contain only the observed prefix.

Table 1: Raw CSV schema. Training and test files share these fields; in the test set, *POLYLINE* contains only the observed initial trajectory.

For an observed prefix

$$P_i = ((\lambda_{i1}, \phi_{i1}), \dots, (\lambda_{iL_i}, \phi_{iL_i})), \quad (1)$$

we predict the complete trip’s final point (λ_i^*, ϕ_i^*) . Internally, every model uses the target order [END_LONG, END_LAT]; submission files convert this to TRIP_ID,LATITUDE, LONGITUDE. No external geographic data are used.

Population or representation	Rows	Features
Raw training trajectories	1,710,670	9
Competition test prefixes	320	9
Retained training trajectories	1,672,901	8 raw fields
Validation training matrix (default)	1,338,320	541
Validation holdout matrix (default)	334,581	541
Final training matrix (default)	1,672,901	541
Final training matrix (CatBoost)	1,672,901	36

Table 2: Populations and cached representations used in the experiments.

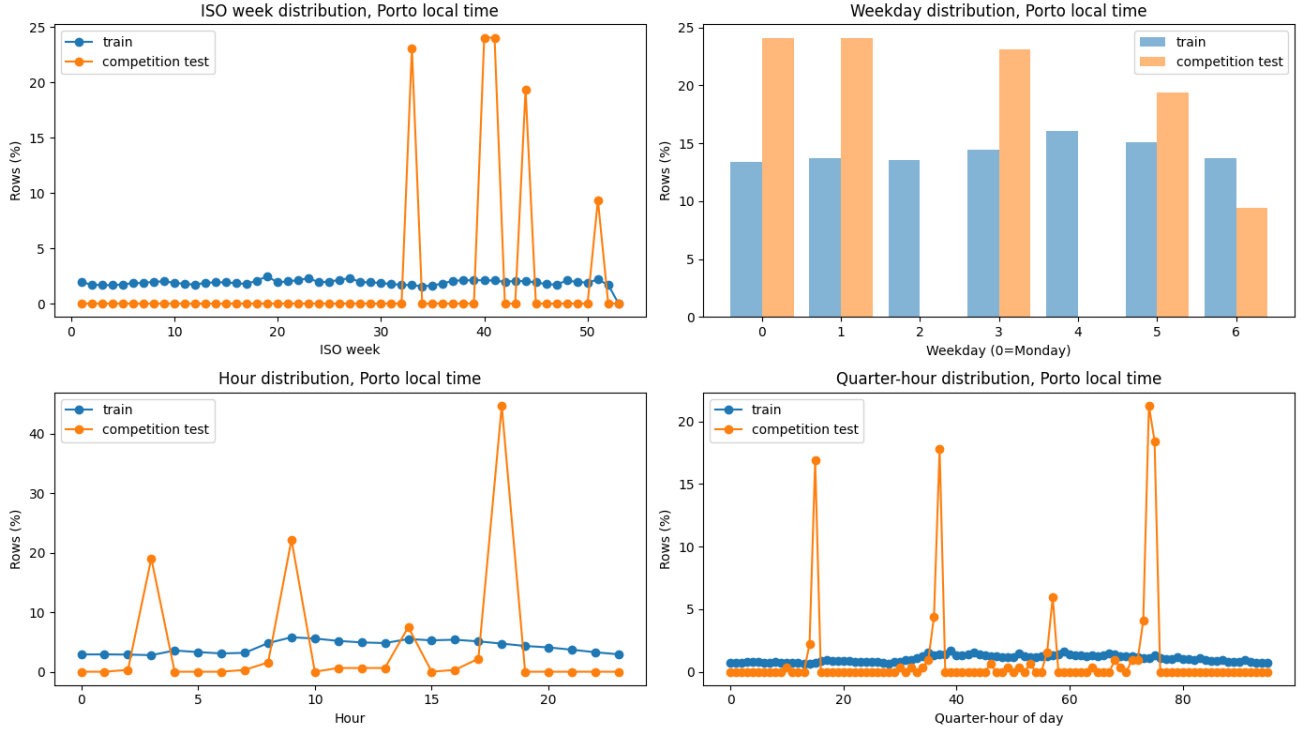


Figure 1: Temporal distributions of the training trips and the 320 competition prefixes in Porto local time. The competition inputs are concentrated in later ISO weeks and a small set of observation times, motivating calendar features and caution when transferring results from a shuffled validation split.

2.2 Cleaning and validation split

We parse each polyline and remove incomplete, invalid, or shorter-than-two-point trajectories. We then apply two conservative training-only checks: the first and final coordinates must fall inside a broad region around Porto, and GPS streams longer than four hours are removed only when they remain within 20 km of their start. These steps remove 36,518 invalid or incomplete rows, 1,109 spatial outliers, and 142 duration artifacts, leaving 1,672,901 trajectories (97.792% of the raw data).

Before feature construction, we create a shuffled 80/20 split with seed 42. The resulting validation design contains 1,338,320 training rows and 334,581 holdout rows. Encoders, frequency statistics, and coordinate scaling are fitted on the training partition and then applied unchanged to the holdout. For the final submissions, the same transformations are refitted on all retained trajectories.

2.3 Competition-aligned synthetic prefixes

The models must learn from complete training trajectories but predict from partial test trajectories of very different lengths. We therefore generate one synthetic prefix per retained row in each partition:

1. sample a desired length L from the 320 supplied test-prefix lengths;
2. select a training trajectory containing more than L points;
3. use its first L points as input and its final point as target; and
4. assert that the destination point is not present in the prefix.

The supplied test trajectories were available during the competition, whereas their destinations and public/private membership were hidden. Matching only their observable length distribution is therefore a competition-aligned use of the test inputs, not target leakage. It produces synthetic prefixes with a

median length of 26 points and a 95th percentile of 152 points, close to 26.5 and 152.15 in the test set. A uniform cut-point strategy would instead produce much shorter examples, especially in the upper tail (Figure 2). Seeds 42 and 43 generate the validation-training and validation-holdout prefixes; seed 142 generates a distinct full-data sample for final fitting.

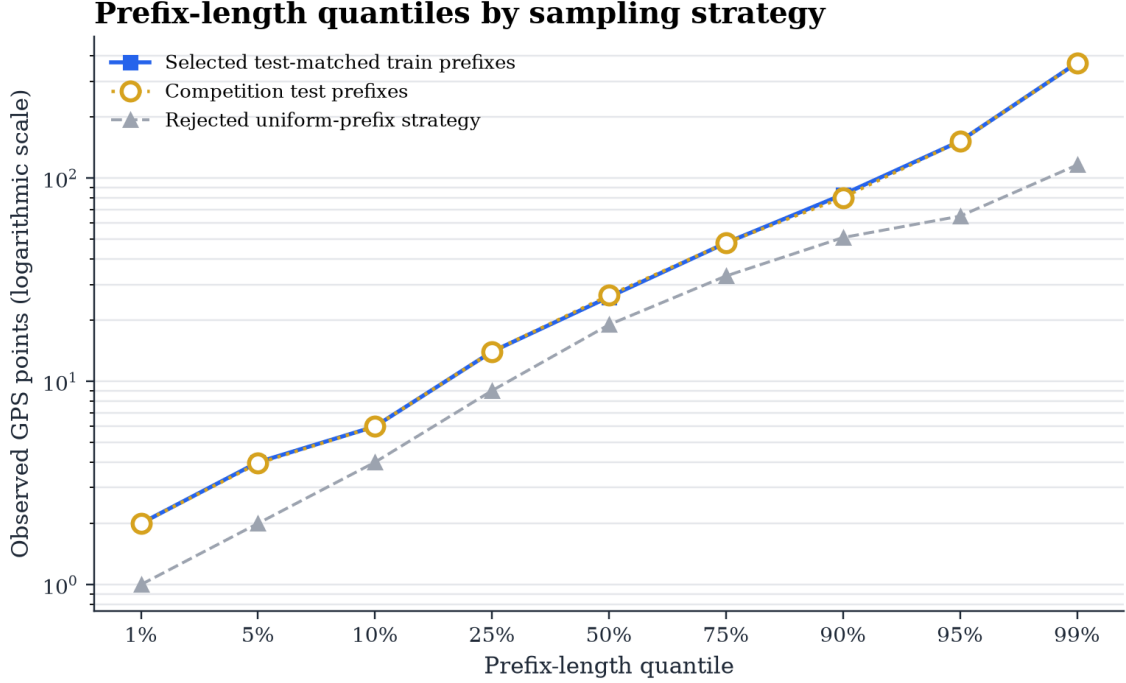


Figure 2: Observed prefix-length quantiles. The selected synthetic strategy follows the supplied test-prefix distribution much more closely than uniform cutting. The vertical axis is logarithmic.

2.4 Fixed-width representation

Each variable-length prefix becomes a fixed-width feature row. We retain the first five and last five observed points in chronological order, giving 20 standardized coordinate features. Short prefixes are padded with their edge coordinate. We add 12 numerical features: caller missingness and log frequency, prefix length, local week, weekday, 15-minute quarter of day, and sine/cosine encodings of the three time cycles.

The default representation one-hot encodes `CALL_TYPE`, `ORIGIN_STAND`, and `TAXI_ID`. Its 541 columns decompose as

$$20 \text{ GPS} + 12 \text{ numeric} + 3 \text{ call type} + 64 \text{ stand} + 442 \text{ taxi} = 541. \quad (2)$$

High-cardinality `ORIGIN_CALL` is represented by missingness and log frequency rather than one-hot encoding. CatBoost receives the same 32 numerical columns plus four native categorical variables—`CALL_TYPE`, `ORIGIN_CALL`, `ORIGIN_STAND`, and `TAXI_ID`—for 36 columns in total. This gives all models the same spatial and temporal information while allowing CatBoost to use its native categorical treatment.

3 Models

We compare seven standalone regressors and one prediction-level ensemble. Linear Regression, Ridge, KNN, Decision Tree, and Random Forest use scikit-learn [8]; XGBoost and CatBoost fit one regressor

per coordinate. Except for CatBoost’s native categorical representation, every standalone model uses the same 541 input features.

3.1 Linear and neighbourhood baselines

Linear Regression and Ridge. Ordinary least squares provides a global additive baseline with an intercept and unconstrained coefficients. Ridge Regression adds an ℓ_2 penalty to stabilize the high-dimensional one-hot design [6]; the selected value is $\alpha = 316.23$. Their similar results test whether the main limitation is coefficient variance or the absence of nonlinear spatial and categorical interactions.

K-nearest neighbours. KNN predicts a destination by averaging nearby training targets [5]. The accepted model uses brute-force Manhattan distance, $k = 256$, and inverse-distance weighting on standardized features. Because a full search is expensive at this scale, tuning and promotion use prefix-length-stratified samples; its validation value is therefore reported separately from the full-holdout evaluations.

3.2 Tree-based models

Decision Tree. The single-tree model supplies a nonlinear control [3]. Its selected configuration has depth 20, minimum split size 150, and minimum leaf size 20. These node-size constraints regularize a tree that would otherwise memorize sparse taxi and stand indicators, but one axis-aligned partition remains sensitive to sampling variation.

Random Forest. Random Forest reduces that variance by averaging randomized trees [2]. We select 700 bootstrap trees, depth 100, 50% feature subsampling, and minimum split and leaf sizes of 5. A slightly better validation mean was available, but the retained configuration reduced the 90th-percentile error by 2.447 m and trained about twice as fast in the recorded search. This model becomes our strongest standalone system.

XGBoost. XGBoost fits a regularized sequence of trees to coordinate residuals [4]. Longitude and latitude use depth-14 GPU models with learning rate 0.05, 90% row subsampling, and 80% column subsampling. Early-stopped validation searches select different regularization and round counts for the two targets; full-data calibration then uses 625 rounds for longitude and 836 for latitude.

CatBoost. CatBoost combines boosting with native treatment of high-cardinality categorical variables [9]. Its two GPU regressors use depth-10 symmetric trees, learning rate 0.03, ℓ_2 leaf penalty 12, and Bayesian bootstrap. Full-data calibration selects 3,207 iterations for longitude and 9,778 for latitude. This model tests whether native caller, stand, and taxi identities improve on one-hot encoding.

3.3 Random Forest–XGBoost ensemble

Random Forest is the most accurate standalone model, while XGBoost offers a different residual pattern and a lower all-trip 90th-percentile error. We combine their frozen coordinate predictions with one convex weight shared by longitude and latitude:

$$\hat{\mathbf{y}}(w) = w\hat{\mathbf{y}}_{\text{RF}} + (1 - w)\hat{\mathbf{y}}_{\text{XGB}}. \quad (3)$$

The final weight is

$$\hat{\mathbf{y}} = 0.8139921906 \hat{\mathbf{y}}_{\text{RF}} + 0.1860078094 \hat{\mathbf{y}}_{\text{XGB}}. \quad (4)$$

Using one shared scalar keeps the ensemble deliberately low-capacity: XGBoost can correct part of the Random Forest error without introducing a coordinate-specific stacker that would be easier to overfit.

4 Experimental design

4.1 Evaluation metric

For a prediction $(\hat{\lambda}_i, \hat{\phi}_i)$ and destination (λ_i, ϕ_i) , we compute the Haversine distance

$$a_i = \sin^2\left(\frac{\phi_i - \hat{\phi}_i}{2}\right) + \cos(\hat{\phi}_i) \cos(\phi_i) \sin^2\left(\frac{\lambda_i - \hat{\lambda}_i}{2}\right), \quad (5)$$

$$d_i = 2R \operatorname{atan2}\left(\sqrt{a_i}, \sqrt{1 - a_i}\right), \quad R = 6371 \text{ km}, \quad (6)$$

$$\mathcal{L}_{\text{hav}} = \frac{1}{n} \sum_{i=1}^n d_i. \quad (7)$$

Mean Haversine distance is the selection and competition metric. Median and 90th-percentile distances are secondary diagnostics for the heavy error tail. Most estimators optimize squared coordinate error internally, so all model decisions are made after recombining longitude and latitude and evaluating the geodesic metric.

4.2 Model selection and final fitting

All standalone models share the 80/20 synthetic-prefix holdout and the same feature definitions. Linear Regression and Ridge use exhaustive finite grids; Decision Tree uses Optuna’s tree-structured Parzen estimator [1]; Random Forest uses a reviewed finite grid; and KNN uses sampled search and promotion cohorts. XGBoost and CatBoost use target-specific catalogues with early stopping, followed by a full-validation refit of the selected longitude/latitude pair. Mean distance is primary, with tail error and runtime used to break practically negligible ties.

After selection, we rebuild the feature transformations on all 1,672,901 retained trajectories, calibrate the final boosting iteration counts where required, fit the accepted configuration, and predict the 320 test prefixes. The final synthetic matrix uses prefix-sampling seed 142 to create a deterministic stream distinct from the validation samples; stochastic estimator seeds remain 42 where applicable.

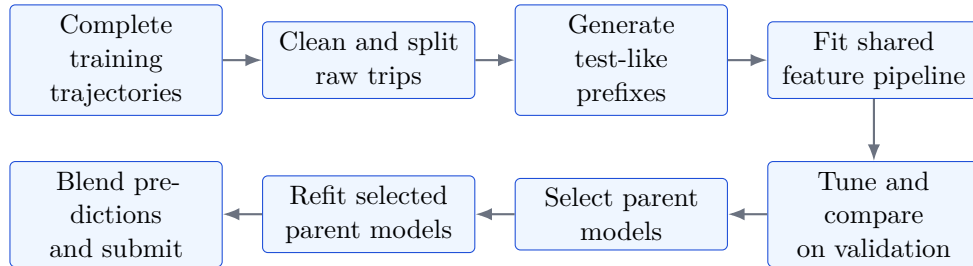


Figure 3: Shared training, selection, and final-fitting workflow. The competition labels do not enter model selection or ensemble-weight estimation.

4.3 Grouped ensemble selection

The Random Forest and XGBoost validation predictions are aligned one-to-one with the same targets. Synthetic sampling can draw the same source trajectory more than once, so we recover the source-trip group and use five repeats of five-fold `GroupKFold`. In each fold, the weight in Equation (3) is learned on four folds and evaluated on held-out trip groups. This produces 25 out-of-fold comparisons with Random Forest.

We promote the blend only if at least four of five repeat means are positive and the trip-group robust 95% interval for the mean gain is strictly above zero. After this gate passes, we estimate one deployable weight from all 334,581 aligned validation rows and apply it to the already-finalized parent submissions. The local public and private labels are used only after the weight is frozen.

4.4 Reported populations and reproducibility

Validation results measure performance on synthetic prefixes drawn from the training year. Local competition results use the released solutions: 160 rows marked public, 160 complementary private rows, and all 320 rows together. We rank models by the local public mean because it matches the competition-facing comparison, while the validation estimate remains the basis for model selection. Runtime values describe the recorded workflow rather than a hardware-normalized benchmark: linear and scikit-learn tree models ran on CPU, XGBoost and CatBoost on GPU, and KNN on sampled cohorts.

The workflow is reproducible from the shared builder `code/build_preprocessed_datasets.py`, the model notebooks, the cached preprocessing archives, and the selected-parameter artifacts. Each notebook separates tuning, validation refit, and final fit so expensive stages can be rerun deliberately. The report itself is regenerated without retraining by running `make` in `report/`; its asset script re-scores the saved submissions and rebuilds the figures and tables.

5 Results

5.1 Random Forest is the strongest standalone model

Table 3 summarizes every final approach. The Random Forest–XGBoost blend achieves the best local public mean, 2.191 km. Random Forest follows at 2.206 km and is the best standalone model; XGBoost is third at 2.276 km. The same three systems lead on the private and all-trip means.

Rank	Model	Public	Private	All	Validation	Fit (s)
1	Ensemble [†]	2.191	2.500	2.346	2.113	–
2	Random Forest	2.206	2.514	2.360	2.119	25,546.1
3	XGBoost	2.276	2.527	2.402	2.215	130.4
4	Decision Tree	2.375	2.732	2.553	2.328	180.3
5	CatBoost	2.390	2.565	2.478	2.297	473.0
6	Ridge	2.665	3.107	2.886	2.888	28.1
7	Linear Regression	2.668	3.099	2.883	2.891	53.1
8	KNN [‡]	3.351	3.568	3.460	3.566	0.577

[†]The ensemble validation value is the mean of five repeated grouped cross-fits; the blend adds no new base-model fit.

[‡]KNN validation and fit time use sampled promotion cohorts. Other standalone validation values use the complete shared holdout. Recorded fit times span CPU and GPU runs.

Table 3: Final model comparison. Distances are mean Haversine errors in kilometres; lower is better. Public and private each contain 160 trips.

The public ranking in Figure 4 makes the top-level separation clear: the ensemble is first, Random Forest is the strongest standalone model, and XGBoost is the next-best complementary system. The ensemble’s 15.1 m advantage over Random Forest is nevertheless small relative to their roughly 2.2 km mean errors, so the chart should be read as a ranking rather than evidence of a large practical gap.

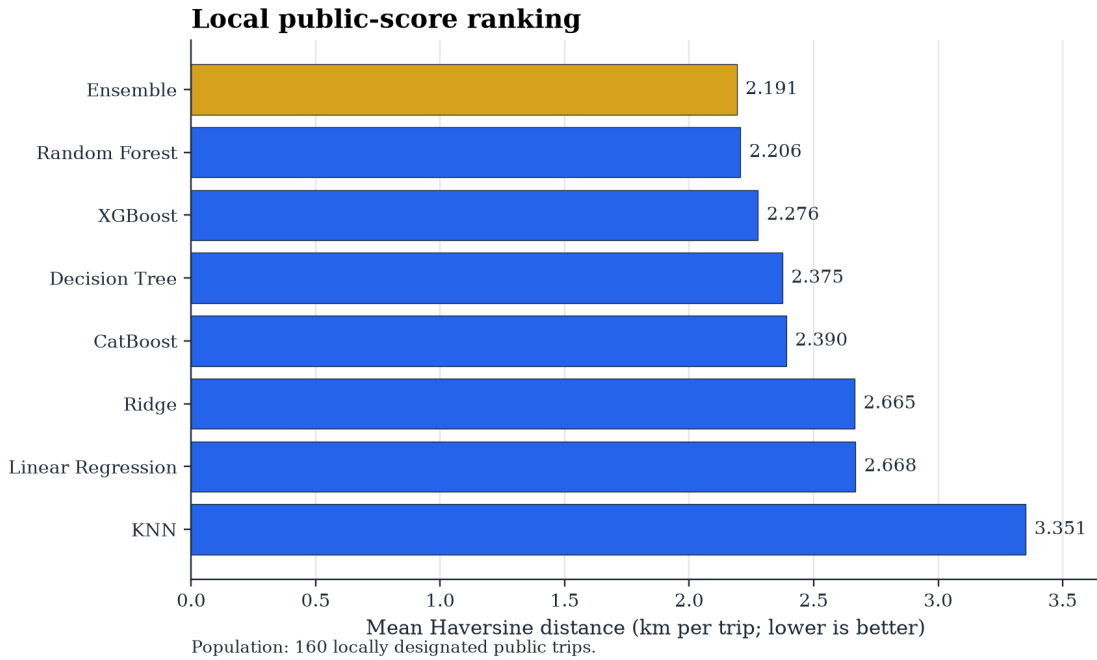


Figure 4: Local public-score ranking on the 160 trips designated public. Bars show mean Haversine error in kilometres; lower is better, and gold highlights the promoted ensemble.

The nonlinear tree families clearly outperform the linear and neighbourhood baselines. Decision Tree improves the public mean by about 293 m relative to Linear Regression, and bagging reduces it by a further 168 m. Ridge and Linear Regression are almost tied, showing that regularization does not overcome the missing nonlinear interactions. KNN ranks last: Manhattan distance in a 541-dimensional mixed spatial and one-hot space is not an effective global notion of route similarity at this scale.

The lower ranks depend on the evaluated subset. CatBoost is better than Decision Tree on validation, private mean, and all-trip mean, but Decision Tree is 15.8 m better on the public subset. Ridge is 2.9 m better than Linear Regression publicly, while Linear Regression is slightly better on the private

and all-trip means. These small reversals should not be interpreted as stable superiority between close models.

5.2 Validation and local scoring tell a consistent top-level story

On the full standalone holdout, Random Forest scores 2.119 km, followed by XGBoost at 2.215 km, CatBoost at 2.297 km, Decision Tree at 2.328 km, Ridge at 2.888 km, and Linear Regression at 2.891 km. KNN’s 3.566 km value comes from its sampled promotion cohort and is not directly comparable. The grouped cross-fitted ensemble reaches 2.113 km. Thus validation and local competition scoring agree on the central conclusion: Random Forest is the strongest base model and XGBoost is the most useful complementary model.

Random Forest’s advantage comes with substantial CPU cost. Its accepted validation refit took 25,546 s, compared with 130 s for GPU XGBoost, 473 s for GPU CatBoost, and 180 s for Decision Tree. XGBoost therefore provides the best observed accuracy–runtime compromise in this workflow. The approximately $196\times$ Random Forest/XGBoost ratio is operational rather than algorithmic, because the runs use different hardware.

5.3 The ensemble gain is consistent but small

The frozen blend assigns 81.399% weight to Random Forest and 18.601% to XGBoost. It improves on Random Forest in all 25 held-out group folds and in all five repeat means (Figure 5). The mean cross-fitted gain is 6.531 m, with a trip-group robust 95% interval of [4.555, 8.506] m. These results pass the pre-defined promotion gate and support using the blend instead of Random Forest alone.

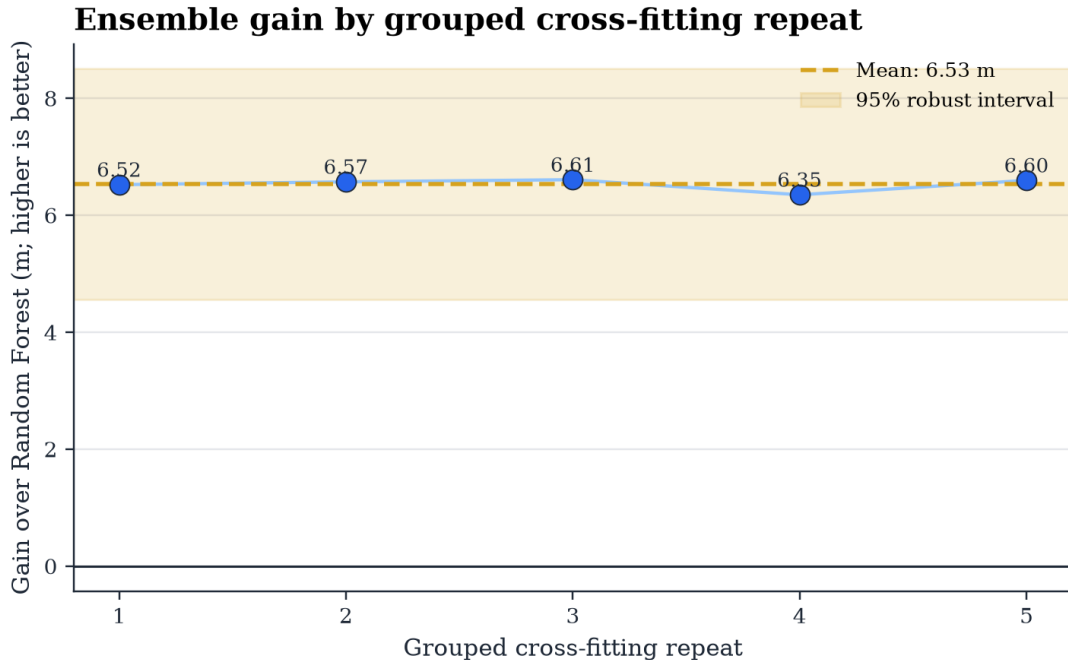


Figure 5: Mean ensemble gain over Random Forest in each repeated five-fold trip-group cross-fit. Positive values favour the blend; the shaded band is the 95% trip-group robust interval for the overall mean gain.

On the local public subset, the ensemble improves the mean by 15.085 m (0.684%) and ranks first at 2.191 km. Across all 320 trips it improves the mean by 14.615 m. The effect is not uniform: Random

Forest has the slightly lower all-trip median (1.236 versus 1.257 km), while the ensemble has the lower 90th percentile (6.222 versus 6.243 km). The blend is therefore an incremental correction that reduces some larger errors rather than a uniformly better prediction on every trip.

6 Discussion, limitations, and next steps

6.1 Interpretation

The main performance gap is between nonlinear tree models and the linear or distance-based baselines. Recent coordinates, time, taxi identity, and dispatch context interact in ways that a global affine model cannot capture. Random Forest benefits most from the large training population: deep randomized trees represent route-specific interactions, while averaging controls the instability of a single Decision Tree. CatBoost’s native categorical handling improves over the single tree on validation and all-trip mean, but it does not surpass Random Forest or XGBoost. This suggests that spatial and temporal interactions matter more than native category processing alone.

The ensemble result is deliberately modest. Random Forest supplies more than 81% of the prediction, and XGBoost acts as a small corrective component. Its gain is positive across every grouped fold, but 6.5 m is less than 0.4% of the Random Forest validation error. We therefore treat the blend as a justified final refinement, not as evidence that a more complicated stacking architecture is required.

The comparison also exposes a practical trade-off. Random Forest is most accurate but expensive on CPU; XGBoost reaches a public error only 70 m higher with a much shorter GPU refit. If training time or iteration speed matters, XGBoost is the more attractive standalone compromise. The recorded timings should not be generalized beyond this hardware configuration.

6.2 Limitations

Our validation design is consistent across model families, but it is not a blind temporal test. Training covers July 2013 to July 2014, whereas the competition trips occur later in 2014. A shuffled holdout does not reproduce that shift, and duplicated trip identifiers or repeated synthetic source trips can weaken row-level independence. Grouped folds address this issue for the ensemble-weight comparison, but the parent models were still selected on the common holdout.

Repeated tuning on one validation set introduces selection optimism, especially for the target-specific XGBoost and CatBoost searches. The spatial and duration filters also use complete-trajectory information and therefore define performance only for the retained population. These choices are reasonable for the competition pipeline, but the validation mean should not be interpreted as a fully unbiased estimate for future Porto trips.

Matching the synthetic prefix-length distribution to the 320 supplied test inputs is not leakage: no test destination or public/private label enters preprocessing. It does, however, specialize the model to the competition’s observed truncation pattern. Performance under a different deployment policy, such as systematically shorter prefixes, would require a separate evaluation.

Finally, the released local solutions make it possible to compare all frozen submissions, but those labels should not be reused for further selection. Public and private means contain only 160 trips each and can reverse close lower-model rankings. We therefore base the ensemble decision on grouped validation and use local scores to describe the final competition outcome.

6.3 Priorities for improvement

Three extensions would strengthen the conclusions more than another broad search on the same holdout:

1. **Grouped temporal validation.** Train on earlier periods and reserve later trip groups for an outer assessment, with a separate inner split for feature, model, and ensemble selection.
2. **Paired error analysis and nested blending.** Retain source-row identifiers, evaluate errors by prefix length and trip context, and generate parent predictions inside each outer fold before learning the ensemble weight.
3. **Standardized reproducibility and compute.** Record one locked software environment and one hardware profile, then report training, prediction, model size, and memory use under comparable conditions.

7 Conclusion

We built a common pipeline that converts complete taxi trajectories into competition-like partial prefixes, engineers a fixed spatial and contextual representation, and compares seven regression families. Random Forest is the strongest standalone model, with mean Haversine errors of 2.119 km on validation and 2.206 km on the local public subset. XGBoost is less accurate but substantially faster in the recorded GPU workflow and contributes complementary errors.

Our final 81.399% Random Forest–18.601% XGBoost blend achieves the best local public score, 2.191 km, and also ranks first on the private and all-trip means. Its grouped cross-fitted gain averages 6.531 m and is positive in every fold, so the improvement is consistent but incremental. The project therefore supports a simple conclusion: careful construction of test-like prefixes and a high-capacity Random Forest provide most of the performance, while a low-capacity blend offers a small final improvement. A grouped temporal evaluation is the most important next step for measuring how well this result transfers beyond the competition split.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. URL <https://arxiv.org/abs/1907.10902>.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- [3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman & Hall, 1984. ISBN 9780412048418.
- [4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [5] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. doi: 10.1109/TIT.1967.1053964.
- [6] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.
- [7] Luís Moreira-Matias, Michel Ferreira, and João Mendes-Moreira. Taxi service trajectory - prediction challenge, ECML PKDD 2015. UCI Machine Learning Repository, 2015. URL <https://archive.ics.uci.edu/dataset/339/taxi+service+trajectory+prediction+challenge+ecml+pkdd+2015>.
- [8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [9] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31*, 2018. URL <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>.